

Lecture 31: Diffusion V

Latent diffusion, video, and what it costs

Some panels are stills from Welch Labs / 3Blue1Brown (2025).

Let us count what we have built

A guided sample at 512×512 costs:

- ▶ 25 sampling steps (DDIM, L29)
- ▶ $\times 2$ forward passes each, for guidance (L30)
- ▶ = 50 passes of a network operating on 786,432 numbers

Every one of those passes runs convolutions and attention over the full image.

Done naively this needs a datacentre for one picture.

And yet Stable Diffusion runs on a gaming laptop, in seconds.

One idea closes that gap - and you already own it.

Outline

Latent diffusion

Flow matching

Video

Where this leaves us

Latent diffusion

The chapter that looked like a detour turns out to be load-bearing.

You already know the punchline

In L23 we closed the autoencoder chapter with a frame titled "*the VAE is inside modern image generators*", and this claim:

The VAE compresses a $512 \times 512 \times 3$ image to $64 \times 64 \times 4$ - about $48\times$ fewer numbers. The diffusion model works in *that*, and the VAE decoder paints the final image.

The VAE did not lose to diffusion. It became the compression layer inside it.

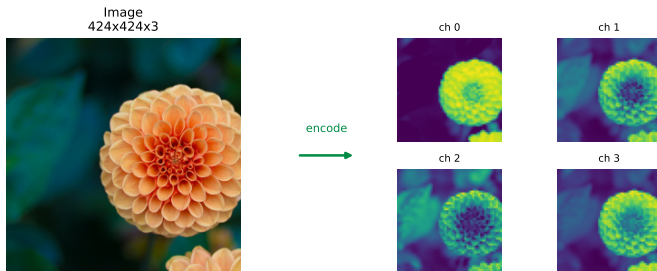
At the time that was a promise. Now you have everything needed to see why it works, and why it is the difference between a research result and a product.

Latent diffusion (Rombach et al., 2022): run the entire process of L27-L30 inside the autoencoder's latent space instead of on pixels.

Nothing about the maths changes. Only where it happens.

What L23 did not tell you: the latent is an image

Latent diffusion: diffuse in a small picture, not in a vector



The latent is still an image.

53x53x4, not a flat code.

Neighbouring latents are still neighbouring patches, so convolutions and attention still mean something in here.

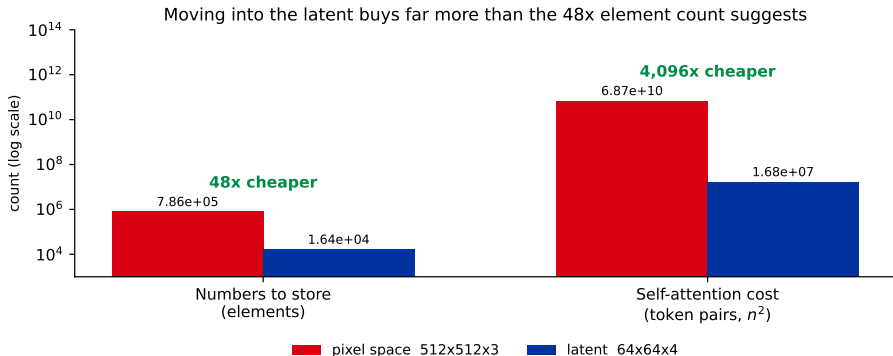
The L22/L23 latent was 2 numbers. This one has spatial layout.

The MNIST autoencoder in L22 squeezed a digit to **two numbers** - a flat code with no geometry. Latent diffusion does not do that.

Its latent is a small **feature map**: neighbouring cells still correspond to neighbouring patches of the picture.

That is the whole trick. Because the latent keeps its spatial layout, convolutions and cross-attention still mean something inside it - so the UNet from L28 works unchanged. Compress to a flat vector and none of it would.

Where the saving actually comes from



Element count drops 48 $\times$. That is the number usually quoted, and it is the *smaller* of the two effects.

Attention cost grows with the **square** of the number of positions. Going from 262,144 positions to 4,096 cuts it by about **4096 $\times$** .

This is why full-resolution pixel-space attention was never really on the table, and why the encoder is not just an optimization but an enabler.

The full picture, and a loose end



Every box is something we built: the UNet in L28, the sampler in L29, the text encoder and cross-attention in L30, the autoencoder back in L22-L23.

The decoder runs **once**, at the very end.

A loose end from L27. The "pure noise" video in our opening slide did not look like television static. Now you know why: it was noise *in the latent*, pushed through the decoder.

Flow matching

If the path is going to be deterministic, why not make it straight?

Straighten the path

DDIM showed the reverse process can be a deterministic trajectory from noise to data. But it is a **curved** one - it inherits its shape from the noise schedule, and curvature is what forces many small steps.

Flow matching (Lipman et al., 2022; Liu et al., 2022) trains the network to predict a **velocity** - which way and how fast to move - rather than the noise, and picks a target path that is as close to a straight line as possible.

A straight path can be crossed in a few large steps without much error. Fewer steps, same quality.

The training loop barely changes: sample a time, interpolate between noise and data, regress onto the target velocity. Still a mean squared error, still no adversarial game.

This is not a historical footnote. Stable Diffusion 3, Flux and Wan are all flow-matching models. If you read a 2025-onward paper, expect this formulation rather than the DDPM one - which is exactly why we derived DDPM first: flow matching is the same skeleton with a different target.

Video

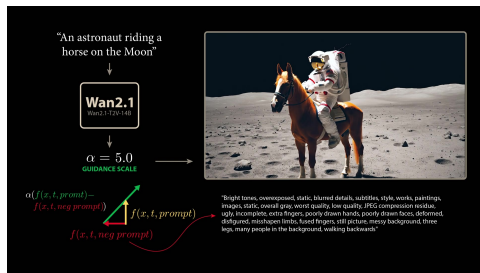
One more axis on the tensor, and remarkably little else.

Video is the same loop with another dimension

An image is $H \times W \times C$. A video is $F \times H \times W \times C$. Noise the whole tensor, denoise the whole tensor.

Everything transfers: the closed-form forward process, $\mathcal{L}_{\text{simple}}$, the sampler, cross-attention on the prompt, classifier-free guidance.

What changes is that the network must attend **across frames** as well as within them - otherwise each frame is a plausible image and the sequence flickers. That is where the engineering effort goes, and it is why video models are so much more expensive.



Welch Labs / 3Blue1Brown (2025)

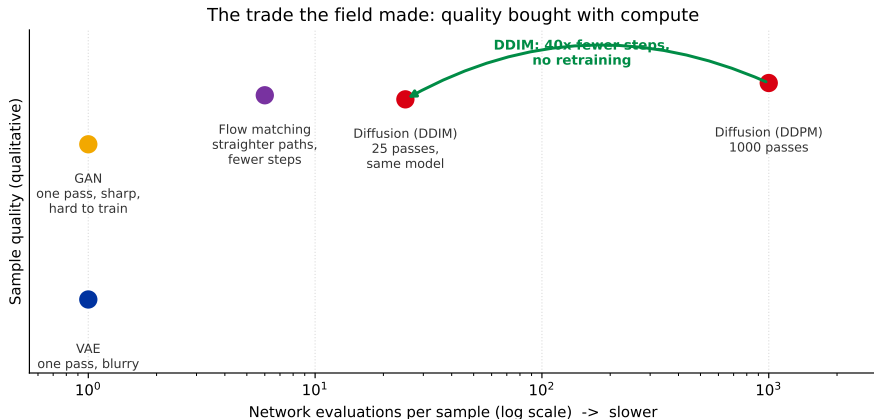
Wan 2.1's shipped negative prompt is a list of everything that tends to go wrong: extra fingers, deformed limbs, cluttered backgrounds, people walking backwards.

A reminder that a large part of production quality is guidance engineering, not architecture.

Where this leaves us

The trade the field made, and the parts that are not technical.

The generative landscape



L23 compared these families on *what they are*. This is the axis that explains the **history**: VAEs and GANs generate in one pass, diffusion needed a thousand and bought quality with them - then DDIM and flow matching won most of that cost back without giving up the quality.

Why images, and not text?

A fair question after a whole chapter on transformers: language models generate one token at a time, image models denoise. Why not diffuse text?

Everything in L27-L28 rests on **continuous** data: add a little Gaussian noise, get something slightly different but still meaningful.

Tokens are **discrete**. "Slightly noisy" is not a word. There is no halfway point between *cat* and *dog*.

There is active work on discrete diffusion and on diffusing in embedding space, and some genuinely promising results. But it is not why your chatbot works.

Continuous data: diffusion. Discrete sequences: autoregression. That split still holds.

What it actually costs

Our thesis has been "diffusion buys quality with compute". It is worth putting a number on that.

Stable Diffusion 1.x was reported by its authors as roughly **150,000 A100-hours** of training - about 17 GPU-years, run in parallel.

Inference is cheap; **training is not**, and it is concentrated in very few hands.

Which is also why open weights matter so much here. Almost nobody can train one of these; a great many people can fine-tune, sample from, and study one.

Everything we did this chapter used a released model or a toy we trained in minutes.

The parts that are not technical

Where the data came from. These models are trained on hundreds of millions of images scraped from the web, largely without the consent of the people who made them. Whether that is fair use is being litigated right now, in several countries, with no settled answer.

What they are used for. The same guidance machinery that puts a tree in a desert puts a real person in a video they were never in. Detection is losing to generation.

Why this is in a technical lecture. You now know how these systems work well enough to build one. That is exactly the point at which the question stops being someone else's.

I am not going to tell you what to conclude. I am telling you not to skip the question.

Watermarking, provenance standards and dataset opt-outs all exist and are all partial. This is unsettled, and you will be working in it.

What we deliberately skipped

Honest map of the edges of this chapter:

- ▶ **Score matching** as its own route to the same models (Song & Ermon, 2019) - we touched the score in L29 and went no further.
- ▶ **Consistency and distillation models**, which push sampling toward one or two steps.
- ▶ **Fine-tuning and control**: LoRA, ControlNet, inpainting.
- ▶ The full **SDE formulation** and its solvers.
- ▶ **Discrete diffusion** for text.
- ▶ Everything about **serving** these models in production.

All of it is readable now. The vocabulary is the barrier, and you have it.

Check yourself

There is no homework for this chapter, so here is how to find out whether it landed. If you can answer these without looking, you have it.

1. Why does the forward process shrink the mean by $\sqrt{1 - \beta_t}$ instead of just adding noise?
2. The training loss is $\|\varepsilon - \varepsilon_\theta\|^2$. Where did the likelihood go?
3. The sampler removes noise, then adds fresh noise. Why is that not a contradiction?
4. DDIM has no noise term at all. Why does *it* not blur?
5. In classifier-free guidance, what does $\alpha = 1$ do, and why is that the neutral value rather than $\alpha = 0$?
6. Conditioning already tells the model the prompt. So what is guidance for?
7. Why does latent diffusion need the latent to be **spatial**?
8. Where does the $4096\times$ come from, given the latent is only $48\times$ smaller?

If you want to go further than answering them: take the sampling loop from L29, get it running on any small dataset, then deliberately break it in the three ways from that lecture's failure slide. You will learn more from the broken runs than the working one.

The chapter, end to end

- L27 Destroy an image in closed form, in a way that reliably lands on $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
- L28 Start from maximum likelihood, follow the algebra, arrive at $\|\varepsilon - \varepsilon_\theta\|^2$.
- L29 Sample - and discover the added noise is keeping the input at the level the network was trained for. Then remove it properly, with DDIM.
- L30 Put a prompt in with cross-attention, and make the model obey it with classifier-free guidance.
- L31 Do all of it in a compressed latent, and it fits on a laptop.

Five lectures ago, a text-to-video model was a black box.

There is no step in it now that you have not seen derived, drawn, or run.

And the thing worth carrying forward is not DDPM specifically. It is the move: **when a distribution is impossible to model directly, find a process that destroys it in a way you control, and learn to run that process backwards.**